

OMI BIND-XOR Core

Official Minimal Execution and Mask Specification

1. Primitive execution model

The execution core contains:

- one unbound coordinate;
- one encapsulation constructor;
- one operation;
- one masking meta-word;
- one escape meta-word;
- four ordered scopes.

The complete executable vocabulary is:

```
NULL  
BIND  
XOR  
META  
ESC  
FS  
GS  
RS  
US
```

`KNOWN` and `UNKNOWN` remain explanatory terms. They do not introduce additional runtime values.

```
UNKNOWN = NULL  
KNOWN   = BIND
```

2. NULL

```
NULL = unbound coordinate
```

`NULL` represents a coordinate at which no active encapsulation has been established.

It may be described as `UNKNOWN`.

`NULL` does not mean false, invalid, absent from reality, or semantically meaningless. It means only that no `BIND` is active at the selected execution coordinate.

3. BIND

`BIND` = only encapsulation

`BIND` establishes a bounded coordinate or relation.

A coordinate enclosed by `BIND` may be described as `KNOWN`.

`KNOWN(x)` iff `BIND(x)` exists

`KNOWN` is therefore not an independent literal. It is the explanatory condition produced by encapsulation.

4. XOR

`XOR` = only operation

`XOR` exposes the distinction between two bounded coordinate conditions.

Using the minimal bound-state encoding:

`NULL` = 0
`BIND` = 1

the operation is:

`XOR(NULL, NULL) = NULL`
`XOR(NULL, BIND) = BIND`
`XOR(BIND, NULL) = BIND`
`XOR(BIND, BIND) = NULL`

The core laws are:

`XOR(x, NULL) = x`
`XOR(x, x) = NULL`

```
XOR(x, y) = XOR(y, x)
XOR(XOR(x, y), y) = x
```

XOR does not interpret the contents of a coordinate. It only exposes, transports, or cancels a distinction.

5. META

```
META = mask for BIND coordinates
```

META is a special meta-word that supplies a mask to the existing **XOR** operation.

It does not introduce another operation.

For a coordinate field **X** and mask **M**:

```
META(M, X) = XOR(X, M)
```

At each coordinate:

```
META position = NULL
    preserve the current coordinate

META position = BIND
    select the coordinate for XOR inversion
```

Therefore:

```
XOR(NULL, NULL) = NULL
XOR(BIND, NULL) = BIND
```

A **NULL** mask leaves a coordinate unchanged.

```
XOR(NULL, BIND) = BIND
XOR(BIND, BIND) = NULL
```

A **BIND** mask toggles the selected coordinate.

META is therefore the selection surface, while **XOR** remains the operation.

6. ESC

ESC = meta-NULL for a selected BIND

ESC is the special mask condition that cancels a selected BIND and returns that coordinate to NULL.

$\text{ESC}(\text{BIND}) = \text{XOR}(\text{BIND}, \text{BIND}) = \text{NULL}$

For an already unbound coordinate:

$\text{ESC}(\text{NULL}) = \text{NULL}$

ESC does not destroy or semantically negate the contents of the relation. It removes the active encapsulation at the selected execution coordinate.

Operationally:

$\text{ESC}(X) = \text{META}(X, X)$

and therefore:

$\text{ESC}(X) = \text{NULL}$

within the declared masked coordinate.

7. Four ordered scopes

The only scopes are:

FS
GS
RS
US

Their ordering is:

```
FS > GS > RS > US
```

Each scope contains exactly four positions.

```
FS = (BIND GS RS US)
GS = (BIND RS US BIND)
RS = (BIND US BIND BIND)
US = (BIND BIND BIND BIND)
```

The scope field develops as follows:

Scope	Direct BIND positions	Subordinate scope positions
FS	1	3
GS	2	2
RS	3	1
US	4	0

The invariant is:

```
direct BIND count + subordinate scope count = 4
```

As the scale descends from **FS** to **US**, subordinate scopes are progressively replaced by direct encapsulation.

US is the completed four-position **BIND** field.

8. Scope registry

The official minimal registry is:

```
((FS . (BIND GS RS US))
 (GS . (BIND RS US BIND))
 (RS . (BIND US BIND BIND))
 (US . (BIND BIND BIND BIND)))
```

Without dotted association notation:

```
((FS (BIND GS RS US))
(GS (BIND RS US BIND))
(RS (BIND US BIND BIND))
(US (BIND BIND BIND BIND)))
```

This registry defines scope ordering and encapsulation structure only. It assigns no external semantic meaning to the values placed within those scopes.

9. Mask application

A four-position field may be masked by `META`.

Example:

```
FIELD = (BIND BIND NULL BIND)
MASK  = (NULL BIND NULL BIND)
```

Applying the mask:

```
META(MASK, FIELD)
=
XOR(
  (BIND BIND NULL BIND),
  (NULL BIND NULL BIND)
)
```

produces:

```
(BIND NULL NULL NULL)
```

Position by position:

```
BIND XOR NULL = BIND
BIND XOR BIND = NULL
NULL XOR NULL = NULL
BIND XOR BIND = NULL
```

The mask does not interpret the field. It only selects which coordinates are preserved or toggled.

10. Execution authority

The execution core is limited to:

```

BIND
    encapsulate a coordinate

XOR
    expose or cancel distinction

META
    select coordinates for XOR

ESC
    cancel a selected BIND to NULL

FS GS RS US
    order the four-position scope field
```

Interpretation remains outside the core.

The framework does not decide whether a coordinate represents:

```

truth
falsity
meaning
identity
ownership
acceptance
authority
knowledge in the human sense
```

It records only bounded coordinate conditions and their exact transformations.

11. Official compact lock

```

UNKNOWN = NULL
KNOWN   = BIND

NULL = unbound coordinate

BIND = only encapsulation
```

XOR = only operation

META = mask selecting coordinates for XOR

ESC = META condition that cancels a selected BIND to NULL

FS = (BIND GS RS US)

GS = (BIND RS US BIND)

RS = (BIND US BIND BIND)

US = (BIND BIND BIND BIND)

12. Canonical statement

The OMI BIND-XOR core is a bounded coordinate execution model. `NULL` is its unbound coordinate and may be described as `UNKNOWN`. `BIND` is its sole encapsulation constructor, and a bound coordinate may be described as `KNOWN`. `XOR` is the sole operation and exposes or cancels distinctions without interpreting their contents. `META` supplies the coordinate mask through which `XOR` is applied, while `ESC` is the special cancellation mask that returns a selected `BIND` to `NULL`. Execution is organized through four ordered scopes—`FS`, `GS`, `RS`, and `US`—each containing four positions and progressively replacing subordinate scopes with direct encapsulation until `US` reaches complete four-position closure. Semantic interpretation remains entirely outside the execution core.